

Automatización de Entornos Multinodo: Docker

Compose para Ciberseguridad

Docker Compose es una herramienta esencial que simplifica el desarrollo, las pruebas y la orquestación de aplicaciones que constan de múltiples servicios interconectados. Permite a los profesionales de ciberseguridad definir, gestionar y reproducir entornos complejos (ej., un *frontend*, una base de datos, y un servicio de *caching*) utilizando un único archivo declarativo en formato **YAML**.

I. Fundamentos de Docker Compose

1. ¿Qué es Docker Compose?

Docker Compose es una herramienta que facilita la definición y ejecución de aplicaciones de **múltiples contenedores** de forma rápida y sencilla. Su funcionalidad se define en un archivo **YAML**.

- **Naturaleza Declarativa:** Permite definir de forma declarativa todos los componentes de una aplicación: los servicios (contenedores), las redes y los volúmenes.
- **Gestión Unificada:** Una vez definido el archivo, un solo comando es suficiente para iniciar, detener y gestionar toda la aplicación y sus contenedores de forma conjunta.
- **Evolución:** Inicialmente, fue una herramienta independiente escrita en Python (**docker-compose**). Actualmente, su funcionalidad se integra como un *plugin* en el cliente de Docker (accesible con **docker compose** sin guion).
- **Consistencia:** Es ideal para el desarrollo, ya que garantiza que todos los desarrolladores que trabajan en el proyecto tengan un **entorno de desarrollo consistente**.

2. Comandos más Utilizados de Docker Compose

A diferencia del `docker` CLI, que trabaja directamente con contenedores y recursos, `docker compose` trabaja con el concepto de **servicio**.

Comando	Equivalente en <code>docker</code> CLI	Propósito
<code>docker compose up</code>	Similar a <code>docker run</code>	Crea y arranca todos los contenedores y redes definidos en el archivo YAML. Por defecto, lo hace de forma interactiva (adjuntando <i>logs</i>).
<code>docker compose up -d</code>	N/A	Crea y arranca los contenedores en modo <i>detached</i> (segundo plano).
<code>docker compose down</code>	<code>docker stop + docker rm + docker network rm</code>	Detiene y elimina los contenedores, así como las redes que haya creado.
<code>docker compose stop</code>	<code>docker stop</code>	Detiene los servicios sin eliminar los contenedores o las redes.

<code>docker</code> <code>compose ps</code>	<code>docker ps</code>	Muestra el estado de los servicios (no de los contenedores).
<code>docker</code> <code>compose logs</code> <code>[servicio]</code>	<code>docker logs</code> <code>[contenedor]</code>	Muestra los <i>logs</i> de un servicio específico.
<code>docker</code> <code>compose run</code> <code>[servicio]</code>	<code>docker run</code> <code>[contenedor]</code>	Permite ejecutar un comando dentro de un servicio (equivalente a un contenedor).

Todos estos comandos se ejecutan desde el mismo directorio donde se encuentra tu archivo `docker-compose.yml`. La sintaxis moderna es `docker compose [COMMAND]`.

1. Construir e Iniciar la Aplicación

Este es el comando más importante, ya que inicia todo el ciclo de vida:

Comando	Sintaxis	Descripción
up	<code>docker compose up -d</code>	Construye las imágenes (si es necesario), crea redes y volúmenes, e inicia todos los servicios definidos. El flag <code>-d</code> inicia en modo <i>detached</i> (segundo plano).
build	<code>docker compose build</code>	Solo construye o reconstruye los servicios que tienen una instrucción <code>build</code> definida en el YAML, sin iniciarlos.

2. Detener y Limpiar la Aplicación

Comando	Sintaxis	Descripción
stop	<code>docker compose stop</code>	Detiene los contenedores en ejecución, pero no los elimina . Puedes reiniciarlos más tarde con <code>docker compose start</code> .
down	<code>docker compose down</code>	Detiene y elimina los contenedores, las redes por defecto y los volúmenes (si se usa el flag <code>-v</code>). Es el comando de limpieza más común.
rm	<code>docker compose rm</code>	Elimina los contenedores detenidos (similar a <code>docker rm</code>).

3. Inspeccionar y Ver Logs

Comando	Sintaxis	Descripción
ps	<code>docker compose ps</code>	Lista los contenedores que forman parte de la aplicación, mostrando su estado (<code>running</code> , <code>exited</code> , etc.).
logs	<code>docker compose logs -f</code>	Muestra la salida de logs de todos los servicios. El flag <code>-f</code> sigue la salida en tiempo real.

exec	<code>docker compose exec [SERVICE] bash</code>	Ejecuta un comando dentro de un contenedor en ejecución (por ejemplo, iniciar una sesión <code>bash</code> en el servicio web).
-------------	---	---

4. Comandos de Soporte

Comando	Sintaxis	Descripción
config	<code>docker compose config</code>	Valida el archivo YAML de Compose y muestra la configuración final generada (útil para depuración).
pull	<code>docker compose pull</code>	Descarga las imágenes de los servicios definidos en el YAML antes de iniciar la aplicación.

Uso: Estructura del Archivo `docker-compose.yml`

```
version: '3' # Version of the Docker Compose file format

services: # Define the services/containers of your application
  service1: # Name of the service/container
    image: image1:tag # Docker image to use for this service
    ports: # Ports to expose
      - "host_port:container_port" # Format: host_port:container_port
    volumes: # Mount volumes
      - "host_path:container_path" # Format: host_path:container_path
    environment: # Environment variables
      - VARIABLE_NAME=value # Format: VARIABLE_NAME=value

  service2:
    image: image2:tag
    # Additional configurations for service2...

networks: # Define networks
  network1: # Name of the network
```

El archivo YAML es la "receta" de tu aplicación. El esquema de arriba muestra su estructura básica:

Sección	Sintaxis/Ejemplo	Propósito
version	<code>'3'</code>	Especifica la versión del formato de archivo Compose (necesario).
services	<code>service1: / service2:</code>	Define los contenedores individuales de tu aplicación (ej. <code>web</code> , <code>db</code> , <code>cache</code>).
image	<code>image1:tag</code>	Especifica la imagen de Docker a partir de la cual construir el servicio.
ports	<code>host_port:containe r_port</code>	Mapea puertos del contenedor al host (como <code>-p</code> en <code>docker run</code>).
volumes	<code>host_path:containe r_path</code>	Monta volúmenes o <i>bind mounts</i> para persistencia (como <code>-v</code> en <code>docker run</code>).
environm ent	<code>VARIABLE_NAME=valu e</code>	Establece variables de entorno para el contenedor (como <code>-e</code> en <code>docker run</code>).
networks	<code>network1:</code>	Define redes personalizadas para que los servicios se comuniquen entre sí.
volumes	<code>volume1:</code>	Define volúmenes persistentes que pueden ser usados por los servicios.

Archivo `docker-compose.yml`

```
version: "3"
services:
  web:
    build: .
    image: visits
    environment:
      - VAR1=value1
      - TAG
    ports:
      - "5000:5000"
    depends_on:
      - redis
    networks:
      - backend

  redis:
    image: "redis:${TAG}"
    networks:
      - backend

networks:
  backend:
```

Explicación Detallada de la Estructura

1. `version: "3"`

- **Propósito:** Especifica el formato del archivo Compose (versión 3).

2. `services:`

Define los dos contenedores de la aplicación: `web` y `redis`.

Subsección `web`: (La Aplicación Flask)

Directiva	Valor en el Ejemplo	Explicación Específica
build:	.	Indica que Docker debe construir la imagen del servicio web utilizando un Dockerfile que se encuentra en el directorio actual (.).
image:	visits	Asigna un nombre a la imagen que se creará a partir del Dockerfile (ej. visits:latest).
environment:	- VAR1=value1	Establece la variable de entorno VAR1 con el valor value1 dentro del contenedor web.
	- TAG	Indica que la variable de entorno TAG debe ser tomada del entorno del host (la máquina que ejecuta docker compose up) y pasada al contenedor web.
ports:	- "5000:5000"	Mapea el puerto 5000 del contenedor al puerto 5000 del <i>host</i> .
depends_on:	- redis	Asegura que el servicio redis se inicie antes que el servicio web .

Subsección **redis**: (El Servidor de Caché)

Directiva	Valor en el Ejemplo	Explicación Específica
image:	"redis:\${TAG}"	Indica que se descargará la imagen de Redis, y el tag (versión) se define dinámicamente utilizando la variable de entorno TAG del <i>host</i> .

Conclusión sobre la Variable TAG

El uso de la variable `${TAG}` es el aspecto más interesante de este archivo:

- **En el servicio web:** La variable **TAG** se pasa al contenedor web (aunque no se use directamente en el código de Python que mostraste, podría usarse en el Dockerfile o en scripts de *entrypoint*).
- **En el servicio redis:** La variable **TAG** se utiliza para especificar la versión exacta de Redis a utilizar (ej. si ejecutas `TAG=6.2 docker compose up`, se usará la imagen `redis:6.2`).

Esto hace que la configuración sea más flexible y reutilizable para diferentes entornos o versiones.

Explicación de las Redes Añadidas

Se han añadido dos secciones principales relacionadas con redes: una sección de definición global y la asignación de esa red a cada servicio.

1. Definición de la Red (Sección Global Inferior)

`networks:`

`backend:`

- **Lugar:** Esta sección se encuentra en el nivel superior del archivo (al mismo nivel que `version` y `services`).
- **Propósito:** Define y configura las redes que utilizará la aplicación.
- **Detalle:** Al simplemente nombrar `backend`, Docker Compose crea una nueva **red de tipo *bridge*** (puente) llamada por defecto `[nombre_del_proyecto]_backend`. Todos los servicios asignados a esta red pueden comunicarse entre sí utilizando sus nombres de servicio (ej. el servicio `web` puede usar `redis` como nombre de host, como ya estaba configurado en tu código Python).

2. Asignación de la Red (Dentro de cada Servicio)

YAML

```
# En el servicio 'web'
```

```
networks:
```

```
- backend
```

```
# En el servicio 'redis'
```

```
networks:
```

```
- backend
```

- **Lugar:** Esta sección se encuentra dentro de las definiciones de los servicios individuales (`web` y `redis`).
- **Propósito:** Conecta los contenedores a la red definida globalmente.
- **Detalle:** Al asignar ambos servicios a la red `backend`, se asegura que estén en la misma subred privada y puedan comunicarse de forma segura e interna.

¿Por qué se añade esto?

Aunque Docker Compose crea automáticamente una red por defecto para todos los servicios, definir explícitamente una red (`backend`) es una buena práctica porque:

1. **Aclara la Intención:** Muestra claramente la arquitectura de la aplicación (que los servicios `web` y `redis` pertenecen a una "red *backend*").

2. **Configuración Avanzada:** Permite configurar opciones avanzadas de red (como subredes específicas o *drivers*).
3. **Aislamiento:** Permite que otros contenedores o servicios que no necesitan acceder a esta capa de la aplicación no se conecten accidentalmente a ella.

II. Definición de la Aplicación en YAML

El archivo de configuración por defecto que busca Docker Compose es **docker-compose.yml** (o **.yaml**). Si el archivo tiene un nombre diferente, debe especificarse con el *flag -f* al ejecutar **docker compose**.

1. Estructura Básica del Archivo

El archivo YAML se basa en tres secciones principales: **version**, **services** y **networks**.

- **version: "3":** Define la versión del formato Compose. Aunque ya no es estrictamente obligatorio, se mantiene por tradición.
- **services::** La sección principal donde se definen los contenedores de la aplicación. El término **servicio** es equivalente al concepto de **contenedor**.
- **networks::** (Opcional) La sección donde se definen las redes personalizadas (redes *bridge*).

2. Definición del Servicio Web (**web**)

El servicio **web** representa una aplicación Python que actúa como un contador de visitas, conectándose a un servicio Redis.

- **build: .:** Indica a Compose que debe **construir** la imagen del servicio utilizando el **Dockerfile** que se encuentra en el directorio actual (.). Compose realiza internamente un **docker build**.
- **image: visits:** Asigna la etiqueta (**tag**) **visits** a la imagen construida.

- **ports:** - **"5000:5000"**: Mapea el puerto **5000** del *host* al puerto **5000** del contenedor (donde escucha la aplicación Flask).
- **environment::** Permite inyectar **variables de entorno** en el contenedor. Es posible asignar un valor explícito (**VAR1=value1**) o referenciar una variable definida en la *shell* del *host* (- **TAG**), la cual se inyectará dinámicamente.
- **depends_on:** - **redis**: Define una dependencia. Compose garantiza que el servicio **redis** se levantará y estará disponible **antes** de intentar arrancar el servicio **web**.

3. Definición del Servicio de Base de Datos (**redis**)

El servicio **redis** no se construye localmente, sino que utiliza una imagen de un registro externo.

- **image:** **"redis:\${TAG}"**: Utiliza la imagen oficial de Redis, y su etiqueta (**tag**) se define dinámicamente mediante la variable **\${TAG}**. Si la variable **TAG** no está definida en la *shell* del *host*, Compose fallará o utilizará una cadena vacía, lo que subraya la necesidad de definir variables como **export TAG=latest**.

4. Comunicación Interna y Nombres de Servicio

La aplicación (**app.py**) se conecta a la base de datos a través del nombre de *host* **redis**.

- **Servidor DNS Interno:** Docker Compose crea un servidor **DNS interno** que resuelve automáticamente el nombre de cada servicio (ej. **web** o **redis**) a su dirección IP interna. Esto permite la comunicación entre contenedores sin necesidad de conocer sus IPs, garantizando la portabilidad.
-

III. Gestión de Redes y Microsegmentación

Docker Compose simplifica la gestión de redes personalizadas, que son fundamentales para la microsegmentación y el aislamiento entre servicios.

1. Redes por Defecto

Si no se define una sección `networks`: explícita, Compose crea una única red `bridge` por defecto (nombrada `[directorio]_default`) y conecta todos los servicios a ella.

2. Definición de Redes Personalizadas

Para un mejor control de la comunicación (microsegmentación), se recomienda definir redes explícitas en la sección `networks`: y asignarlas a los servicios.

- `networks: backend::` Define una red `bridge` personalizada llamada `backend`.

Asignación a Servicios: Ambos servicios, `web` y `redis`, se asignan a esta misma red.

`web:`

```
networks:  
  - backend
```

`redis:`

```
networks:  
  - backend
```

- *Resultado:* Los servicios `web` y `redis` pueden comunicarse entre sí a través de sus nombres dentro de la red `backend` (cumpliendo el requisito de la aplicación), y están aislados de otros servicios en el `host` que no estén conectados a `backend`.
-

IV. Buenas Prácticas y Rendimiento de Compose

Docker Compose no crea recursos nuevos, sino que actúa como una capa de abstracción que utiliza los recursos subyacentes de Docker (`containers`, `networks`, `volumes`) de forma declarativa.

- **Reconstrucción y Caché (`--build`):**
 - Si se hacen cambios en el código fuente (`app.py`), Compose por defecto reutiliza la imagen antigua de la caché si no ha habido cambios en el `Dockerfile`.
 - Para evitar confusiones durante el desarrollo activo, se debe forzar la reconstrucción de la imagen al arrancar el *stack*.

```
docker compose up -d --build
```

- **Gestión del Stack:**
 - **`docker compose up`**: Arranca la aplicación (se puede usar `-d` para *detached*).
 - **`docker compose down`**: Es la forma más limpia de apagar, ya que **detiene y borra** los contenedores y las redes creadas por Compose.
- **Ruta de Ejecución:** Los comandos `docker compose` deben ejecutarse siempre desde el directorio que contiene el archivo `docker-compose.yaml`.

Código de la app.py usado en la práctica

```
import time
import redis
from flask import Flask

app = Flask(__name__)
cache = redis.Redis(host='redis', port=6379)

def get_hit_count():
```

```

retries = 5
while True:
    try:
        return cache.incr('hits')
    except redis.exceptions.ConnectionError as exc:
        if retries == 0:
            raise exc
        retries -= 1
        time.sleep(0.5)

@app.route('/')
def hello():
    count = get_hit_count()
    return 'Número de visitas: {}\n'.format(count)

if __name__ == "__main__":
    app.run(host="0.0.0.0", debug=True)

```

Explicación Rápida del Código

Este código define una aplicación web muy simple utilizando el *framework* **Flask**, cuyo objetivo es **contar cuántas veces se ha accedido a la ruta principal (/)** y mostrar ese número.

1. **Importaciones:** Se importan las librerías `time`, `redis` y `Flask`.
2. **Inicialización de Flask y Redis:**
 - `app = Flask(__name__)`: Inicializa la aplicación Flask.
 - `cache = redis.Redis(host='redis', port=6379)`: Se conecta al servidor Redis. Nota importante: usa `host='redis'`, lo que implica que esta aplicación está diseñada para ejecutarse dentro de un contenedor Docker o en un entorno de red donde el servicio Redis es accesible bajo el nombre de host `redis` (típicamente a través de **Docker Compose** o una red de Docker).
3. **Función `get_hit_count()`:**

- Esta función intenta incrementar el valor de la clave `'hits'` en Redis (`cache.incr('hits')`).
- Incluye una lógica de **reintentos** (`retries = 5`) para manejar fallos temporales de conexión a Redis (por ejemplo, si el contenedor de Redis tarda un poco en iniciarse). Si la conexión falla 5 veces, lanza la excepción y detiene el programa.

4. Función `hello()` (Ruta Principal):

- Esta función se ejecuta cada vez que alguien visita la ruta raíz (`@app.route('/')`).
- Llama a `get_hit_count()` para incrementar y obtener el número de visitas.
- Devuelve una cadena de texto al usuario con el número actualizado.

5. Ejecución:

- `app.run(host="0.0.0.0", debug=True)`: Inicia el servidor Flask, haciéndolo accesible públicamente en el contenedor a través de cualquier interfaz (`0.0.0.0`) y habilitando el modo de depuración.

Seguridad Profunda en Docker: Fortificación del Host, Daemon y Contenedores

En un Máster en Ciberseguridad, la seguridad de **Docker** requiere una estrategia de defensa en profundidad que abarque desde la máquina *host* hasta el proceso individual del contenedor. Este informe detalla los cuatro aspectos clave de la seguridad en Docker: el *host*, el *daemon*, las imágenes, y el *runtime* del contenedor, incluyendo buenas prácticas y herramientas esenciales de escaneo.

I. Aspectos Fundamentales de la Seguridad en Docker

La seguridad de un entorno contenerizado se aborda en cuatro niveles interconectados:

1. **Seguridad del Host:** Protección del sistema operativo anfitrión, cuyo *kernel* es compartido.
2. **Seguridad del Docker Daemon:** Protección del servicio que gestiona la API de Docker.
3. **Seguridad de la Imagen:** Minimización y fortificación de las imágenes en tiempo de construcción.
4. **Seguridad del Contenedor:** Restricción de privilegios y monitorización en tiempo de ejecución.

II. Fortificación del Host y el Daemon

2.1. Seguridad del Host (Sistema Anfitrión)

Dado que los contenedores de Docker comparten el **núcleo (*kernel*)** del sistema anfitrión, una vulnerabilidad o configuración incorrecta en el *kernel* puede ser explotada por un contenedor malicioso para obtener acceso al sistema *host*.

- **Fortificación (*Hardening*):** Es esencial seguir las mejores prácticas para el fortalecimiento del *host*:

- Desactivar servicios innecesarios.
- Mantener el sistema *host* actualizado con parches de seguridad.
- Implementar el **Principio de Privilegio Mínimo** (*Least Privilege Principle*) y controles de acceso.
- Reducir la superficie de ataque del sistema anfitrión.
- **Monitorización y Auditoría:** Implementar soluciones robustas de auditoría, *logging* y monitorización.
 - Monitorizar la actividad de los contenedores, *logs* del sistema y tráfico de red.
 - Detectar comportamientos sospechosos o posibles violaciones de seguridad.

2.2. Seguridad del Docker Daemon

El *Docker Daemon* no es más que una **API REST**. Su protección es vital para evitar el **escape de contenedores**.

- **Riesgo de Escape:** Existen vulnerabilidades que pueden permitir a los atacantes **salir del contenedor** y obtener acceso al sistema anfitrión. Esto puede ocurrir debido a configuraciones incorrectas, vulnerabilidades en Docker, o vulnerabilidades en las aplicaciones.
- **Restricción de Acceso:** El acceso al *Docker Daemon* debe ser **restringido** a usuarios y procesos autorizados.
- **Controles de Seguridad:** Elegir un motor de ejecución seguro (como *containerd* o *CRI-O*) y configurarlo con características de seguridad apropiadas, por ejemplo, siguiendo estándares como **Seccomp**, **AppArmor** o **SELinux**.
- **Red y Firewall:** Configurar correctamente las políticas de red, reglas de *firewall* y la red de contenedores para minimizar el riesgo de acceso no autorizado.

III. Seguridad de la Imagen y Buenas Prácticas de Construcción

La seguridad se inicia en el proceso de construcción. Una imagen limpia y pequeña reduce el vector de ataque.

3.1. Minimización de la Imagen

- **Imágenes Mínimas:** Es crucial la creación de **imágenes pequeñas** para reducir el riesgo de seguridad. Cuantas menos cosas tenga la imagen, menos recursos tendrá un atacante para causar daño.
- **Imágenes Distroless:** Cuando sea posible, es recomendable utilizar imágenes **distroless** o similares. Estas imágenes tienen lo mínimo necesario para la ejecución de la aplicación, lo que reduce drásticamente los vectores de ataque.

3.2. Fortificación del Usuario (USER)

Una de las buenas prácticas más importantes es que la aplicación se ejecute con un usuario con el **mínimo privilegio**.

- **Riesgo de root:** Ejecutar contenedores como **root** es un grave problema de seguridad. Si el servicio es comprometido, el atacante obtendría acceso **root** al contenedor, lo cual a menudo es un requisito para que las vulnerabilidades de escape de contenedor puedan ejecutarse.
- **Instrucción USER:** La instrucción **USER** en el Dockerfile se utiliza para definir el usuario (por nombre o ID) con el que se ejecutará el proceso.
- **Creación de Usuario (Non-Root):** Para cumplir con el principio de privilegio mínimo, se debe crear un usuario de aplicación dedicado (**appuser**) y ejecutar la aplicación con él.
 - **Creación en Dockerfile:** Se utiliza la instrucción **RUN** y el comando **useradd** para crear el usuario.

```
RUN useradd --no-log-init --system --shell  
/usr/sbin/nologin appuser
```

- *El uso de **--no-log-init** y **--shell /usr/sbin/nologin** minimiza las capacidades del usuario.*

- **Establecer Usuario:** La instrucción `USER` debe estar al final del Dockerfile.
`USER appuser`
- **Orden de `USER` (Privilegios):** El orden de `USER` en el Dockerfile es fundamental.
 - Cualquier instrucción después de `USER appuser` se ejecutará con ese usuario.
 - Los comandos que requieren privilegios elevados (ej. `apt update`, `apt install`) **deben ejecutarse con el usuario `root`** y colocarse antes de la instrucción `USER appuser`. Si el usuario no `root` intenta ejecutar `apt update`, la operación fallará por falta de permisos.
- **Gestión de Permisos:** Si el usuario *non-root* necesita acceder a directorios o ficheros específicos, se debe usar `chown` para cambiar el propietario de esos ficheros al nuevo usuario (`appuser`) antes de que el contenedor se ejecute con dicho usuario.

3.3. Comprobaciones de Salud (`HEALTHCHECK`)

- **Disponibilidad (Tercer Pilar de Infosec):** La disponibilidad es un pilar crítico de la seguridad de la información (Confidencialidad, Integridad y Disponibilidad - CIA).
- **Comando `HEALTHCHECK`:** Define un chequeo que Docker ejecuta periódicamente para determinar si el contenedor está funcionando correctamente (es *healthy*) o si necesita ser reiniciado.
 - **Sintaxis:** `HEALTHCHECK [OPTIONS] CMD command`
 - **Comportamiento:** Docker ejecuta el `CMD` especificado en un intervalo dado. Si el comando devuelve un código de salida **cero**, el contenedor se considera saludable. Cualquier otro código de salida positivo indica un error.

- **Ejemplo Práctico:**

```
HEALTHCHECK --interval=5s --timeout=1s --retries=1
```

```
CMD curl -f http://localhost:5000
```

- **--interval=5s:** Intervalo entre chequeos.
- **--timeout=1s:** Tiempo máximo de espera para la respuesta del comando.
- **CMD curl -f http://localhost:5000:** Comando que realiza una petición HTTP a la aplicación (puerto 5000).
- **Estado del Contenedor:** Docker reporta el estado como *unhealthy* (no saludable) o *healthy* (saludable).

El comando **HEALTHCHECK** permite a Docker verificar si un contenedor está funcionando **correctamente** (no solo si el proceso principal está activo).

1. Formas de Uso (**Syntax**)

Forma	Sintaxis	Propósito
Verificación	HEALTHCHECK [OPTIONS] CMD command	Ejecuta un comando dentro del contenedor para verificar su estado.
Deshabilitar	HEALTHCHECK NONE	Deshabilita el <i>healthcheck</i> heredado de la imagen base.
Comando	CMD /bin/check-running o CMD ["curl", "-f", "http://..."]	El comando puede ser una sentencia shell o un array exec (recomendado).

2. Estados de Salud (**Health Status**)

Un contenedor con **HEALTHCHECK** tiene un estado de salud adicional a su estado normal:

Estado Inicial	Transición (Éxito)	Transición (Fallo)
starting (Iniciando)	Pasa a healthy	Tras retries fallos consecutivos, pasa a unhealthy .

3. Códigos de Salida del Comando **CMD**

El código de salida del comando determina el estado de salud:

Código de Salida	Significado	Estado Resultante
0	Éxito	El contenedor está healthy (saludable).
1	Fallo	El contenedor no funciona correctamente (unhealthy).
2	Reservado	No debe utilizarse.

4. Opciones de Configuración (**OPTIONS**)

Opción	Propósito	Valor Por Defecto
--------	-----------	-------------------

--interval=DURATION	Tiempo de espera entre la finalización de un chequeo y el inicio del siguiente.	30s
--timeout=DURATION	Tiempo máximo permitido para que se ejecute una única verificación. Si se supera, se considera fallo.	30s
--retries=N	Número de fallos consecutivos necesarios para que el contenedor se considere unhealthy .	3
--start-period=DURATION	Período de gracia inicial para el <i>bootstrapping</i> (inicialización) del contenedor. Los fallos durante este tiempo no se contabilizan para el límite de retries , pero un éxito termina el período.	0s
--start-interval=DURATION	Tiempo entre chequeos durante el start-period .	5s

5. Características Adicionales

- **Salida de Debug:** Cualquier texto (UTF-8) escrito por el comando en **stdout** o **stderr** se almacena en el estado de salud (máx. 4096 bytes) y puede consultarse con **docker inspect**.
- **Eventos:** Cuando el estado de salud cambia (**starting** -> **healthy**, **healthy** -> **unhealthy**), se genera un evento **health_status**.
- **Límite:** Solo se permite **una** instrucción **HEALTHCHECK** por Dockerfile; la última será la que tenga efecto.

3.4. Escaneo de Vulnerabilidades (**docker scout**)

Debido a que las imágenes contienen no solo el código de la aplicación, sino también todas las utilidades y librerías de la imagen base, el **escaneo continuo de vulnerabilidades** es una práctica obligatoria.

- **Propósito:** Identificar vulnerabilidades comunes (CVEs) en las librerías, paquetes del sistema (**dpkg**, **rpm**, **apk**) y dependencias del lenguaje de la aplicación.
- **Herramienta Docker Scout:** Docker ofrece su propia herramienta de escaneo de seguridad llamada **docker scout**.

- **Sintaxis de Escaneo CVE:**

```
docker scout cves healthcheck
```

- **Funcionamiento:** El escáner identifica el sistema de paquetes, genera una lista de paquetes instalados con sus versiones y los compara con bases de datos públicas de CVEs.

Comandos de **docker scout**:

Comando Disponible	Uso
attestation	Gestiona atestaciones en índices de imágenes.
cache	Gestiona la caché y archivos temporales de Docker Scout.
compare	Compara dos imágenes y muestra sus diferencias (experimental).
config	Gestiona la configuración de Docker Scout.

cves	Muestra los CVE (Vulnerabilidades y Exposiciones Comunes) identificados en un artefacto de software.
enroll	Registra una organización con Docker Scout.
environment	Gestiona entornos (experimental).
help	Muestra información sobre los comandos disponibles.
integration	Comandos para listar, configurar y eliminar integraciones de Docker Scout.
policy	Evalúa políticas contra una imagen y muestra los resultados de la evaluación de políticas (experimental).
quickview	Muestra una visión general rápida de una imagen.
recommendations	Muestra las actualizaciones de imágenes base disponibles y recomendaciones de remediación.
repo	Comandos para listar, habilitar y deshabilitar Docker Scout en repositorios.
version	Muestra la información de la versión de Docker Scout.

IV. Contenido Completo del Dockerfile de Demostración

El siguiente `Dockerfile` se utilizó para las demostraciones de seguridad, incluyendo optimización de caché, instalación de `curl`, la definición del `HEALTHCHECK` y el establecimiento de un usuario *non-root*.

```
FROM python:3.10

RUN apt update && apt upgrade -y && apt install curl -y

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

HEALTHCHECK --interval=5s --timeout=1s --retries=1 CMD curl -f
http://localhost

RUN useradd --no-log-init --system --shell /usr/sbin/nologin
appuser

USER appuser

CMD ["python3", "app.py"]
```

V. Conclusiones Finales

La seguridad en Docker debe ser abordada con una mentalidad de **defensa en profundidad**.

- **Fortificación Total:** Es imprescindible proteger el *host* (parches y *hardening*), asegurar el *Docker Daemon* (restricción de acceso, uso de AppArmor/SELinux), y mantener las imágenes lo más pequeñas y limpias posible.

- **Privilegios Mínimos (USER):** Nunca se deben ejecutar contenedores como **root** en producción. La creación y uso de un usuario *non-root* dedicado es un factor clave para mitigar las vulnerabilidades de escape de contenedor.
- **Disponibilidad (HEALTHCHECK):** La definición de puntos de salud permite a Docker reportar cuando un contenedor no está funcionando como debería.
- **Escaneo Continuo:** El escaneo constante de imágenes con herramientas como **docker scout** o Trivy es una buena práctica para mantener las imágenes libres de vulnerabilidades conocidas.
- **Protección del Daemon:** El *Docker Daemon* debe estar bien protegido y **no expuesto a Internet** para evitar el compromiso del sistema subyacente.

Seguridad en Contenedores Docker: Aplicación del Principio de Mínimo Privilegio

La seguridad de los contenedores se centra en la aplicación estricta del **Principio de Mínimo Privilegio**, limitando los recursos, aislando los espacios de nombres del *host* y eliminando capacidades innecesarias. Un contenedor no es más que un proceso; por lo tanto, las mismas prácticas de seguridad aplicadas a cualquier servicio del sistema son cruciales.

I. Aspectos Clave de la Seguridad en el *Runtime* del Contenedor

Para proteger el sistema anfitrión de un contenedor comprometido, se deben gestionar las siguientes áreas:

1. **Limitación de Recursos (Cgroups):** Limitar el uso de CPU, memoria y otros recursos para evitar que un contenedor malicioso o defectuoso consuma todos los recursos del *host* (riesgo de Denegación de Servicio).
2. **Aislamiento de *Namespaces*:** No compartir los *namespaces* críticos del *host* (red, procesos, etc.) para mantener el aislamiento fundamental de Docker.
3. **Montajes de Sistemas de Ficheros:** Evitar montar el sistema de ficheros del *host* dentro del contenedor, ya que esto otorga un acceso peligroso a la máquina anfitriona.
4. **Modo Privilegiado:** No ejecutar contenedores en modo privilegiado, ya que esto anula todas las protecciones del *kernel* de Linux.
5. **Capacidades (*Capabilities*):** Eliminar o reducir las capacidades por defecto que otorga Docker al contenedor, dejando solo las estrictamente necesarias.

II. Procedimientos Prácticos de Fortificación

2.1. Limitación de Recursos (CPU y Memoria)

El comando `docker run` permite establecer límites de recursos directamente en la CLI, utilizando las funcionalidades de Cgroups de Linux.

- **Sintaxis de Limitación de Recursos:**

```
docker run -it -m [MEMORIA] --cpus [N_CPUS] [IMAGEN]
```

- **-m o --memory:** Limita la **memoria RAM** asignada al contenedor (ej. `1g` para un gigabyte). Si el contenedor sobrepasa este límite, Docker lo mata con un error *Out Of Memory* (OOM).
- **--cpus:** Limita el número de **núcleos de CPU** asignados al contenedor. Si el contenedor excede este límite, Docker no le asigna más CPU, lo que ralentiza el proceso.
- **Deshabilitar OOM Kill (Advertencia):** Para evitar que Docker mate un contenedor por exceder el límite de memoria, se puede usar la opción `--oom-kill-disable`. Sin embargo, esto es una opción de alto riesgo, ya que permite que el proceso continúe a costa de la estabilidad del *host*.

2.2. Peligro de Compartir *Namespaces* y Sistemas de Ficheros

Compartir *namespaces* o montar sistemas de ficheros críticos del *host* anula el aislamiento de seguridad y debe evitarse a toda costa para aplicaciones de uso general.

- **Montaje del Sistema de Ficheros Raíz (/):** Montar el directorio raíz del *host* en el contenedor, especialmente si se ejecuta como *root*, otorga un control casi total sobre el sistema anfitrión.

Ejemplo de MUY mala práctica

```
docker run -it -v /:/host ubuntu
```

- Una vez dentro, si el usuario es *root*, podría usar `chroot` para operar como *root* en el *host*.

- **Montaje del Socket de Docker:** Montar el *socket* del *Docker Daemon* (`/var/run/docker.sock`) dentro del contenedor le da al contenedor acceso completo y con privilegios de *root* al *Docker Daemon* del *host*, permitiéndole crear, matar o modificar cualquier contenedor, o incluso escapar.
- **Conexión a *Namespaces* del Host:** Opciones como `--privileged`, `--network host`, `--pid host`, o `--ipc host` deben ser usadas solo en casos extremos (ej. herramientas de monitoreo o escaneo a nivel de sistema).
 - **Modo Privilegiado (`--privileged`):** Otorga al contenedor todas las capacidades del *kernel* y anula las restricciones, permitiéndole hacer prácticamente cualquier cosa en el *host* (manipular la red, matar procesos, etc.). **Esto debe evitarse siempre.**

2.3. Gestión de Capacidades de Linux (*Capabilities*)

Las **Capacidades de Linux** son privilegios granulares que se asignan a un proceso. Docker asigna un conjunto por defecto a los contenedores, muchos de los cuales no son necesarios para una aplicación web estándar (ej. `CAP_CHOWN`, `CAP_SETUID`, `CAP_NET_BIND_SERVICE`).

- **Principio del Mínimo Privilegio:** La práctica de seguridad recomendada es **eliminar todas las capacidades por defecto** y luego añadir solo las que sean estrictamente necesarias.

1. Verificar Capacidades Actuales (Dentro del Contenedor):

- Se debe instalar un paquete (ej. `libcap2-bin` en Debian/Ubuntu).
- Se inspeccionan las capacidades efectivas (`CapEff`) del proceso PID 1 (el proceso principal del contenedor).

```
grep Cap /proc/1/status
```

- Para decodificar el valor hexadecimal, se utiliza la utilidad `capsh`.

```
capsh --decode=[CapEff_HEX_VALUE]
```

2. **Eliminar Todas las Capacidades:** El *flag* `--cap-drop=ALL` elimina todas las capacidades por defecto.

```
docker run -it --cap-drop=ALL ubuntu
```

Resultado: Incluso como *root*, el contenedor perderá la capacidad de realizar acciones básicas de administración de sistema (ej. `apt update` fallará, y comandos como `chown` no funcionarán).

3. **Añadir Solo las Necesarias:** Si la aplicación falla, se usa `--cap-add` para reintroducir las capacidades mínimas requeridas.

- **Sintaxis Combinada:**

```
docker run -it --cap-drop=ALL --cap-add=CAP_CHOWN  
ubuntu
```

- **Propósito:** En este ejemplo, el contenedor solo recupera la capacidad de cambiar la propiedad de los archivos (`CAP_CHOWN`), mientras que el resto de privilegios peligrosos permanecen eliminados.

2.4. Flags de Docker Run de CPUS y Memoria

Opciones (flags) del comando Docker relacionadas con la **gestión y limitación de recursos de CPU** de un contenedor:

Comando (Flag)	Tipo	Explicación en Castellano

--cpu-period	int	Limita el período del planificador CFS (Completely Fair Scheduler) de la CPU.
--cpu-quota	int	Limita la cuota del planificador CFS (Completely Fair Scheduler) de la CPU.
--cpu-rt-period	int	Limita el período de CPU en tiempo real en microsegundos.
--cpu-rt-runtime	int	Limita el tiempo de ejecución de CPU en tiempo real en microsegundos.
--cpu-shares	int	Comparticiones de CPU (peso relativo, para asignar prioridad).
--cpus	decimal	Número de CPUs (o núcleos) que puede usar el contenedor.
--cpuset-cpus	string	CPUs específicas en las que se permite la ejecución (ej: 0-3, 0, 1).
--cpuset-mems	string	MEMs específicas (nodos de memoria) en las que se permite la ejecución (ej: 0-3, 0, 1).

Opciones (flags) del comando Docker relacionadas con la **gestión y limitación de recursos de memoria** de un contenedor:

Comando (Flag)	Explicación en Castellano
----------------	---------------------------

--memory bytes	Límite de memoria (Memory limit).
--memory-reservation bytes	Límite flexible de memoria (Memory soft limit).
--memory-swap bytes	Límite de swap (memoria más swap). Use '-1' para habilitar swap ilimitado.
--memory-swappiness int	Ajusta la "swappiness" (tendencia al intercambio) de la memoria del contenedor (0 a 100) (predeterminado: -1).

III. Conclusiones y Fortificación del *Runtime*

La fortificación del *runtime* del contenedor es el último bastión de defensa. Cuantos menos permisos, capacidades y acceso a recursos tenga el contenedor, menor será el impacto de una brecha de seguridad.

- **Aislamiento Prioritario:** Evitar a toda costa mapear los *namespaces* del *host* (red, procesos) o montar sistemas de ficheros sensibles.
- **Control de Recursos:** Limitar los recursos (**-m**, **--cpus**) para prevenir ataques de Denegación de Servicio (DoS).
- **Eliminación de Privilegios:**
 - **No root:** Asegurar que el proceso se ejecute con un usuario no privilegiado (**USER appuser**).
 - **No Privilegiado:** Nunca usar el *flag* **--privileged**.
 - **Mínimas Capacidades:** Usar **--cap-drop=ALL** como regla general y solo añadir (**--cap-add**) las capacidades que sean indispensables.

Auditoría de Seguridad en Docker: Uso de Docker Bench for Security y CIS Benchmarks

La implementación de buenas prácticas de seguridad en un entorno Docker es compleja, abarcando el *host*, el *daemon* y los contenedores. Para asegurar el cumplimiento y la mitigación de riesgos, la **auditoría frecuente y automatizada** del sistema es esencial. **Docker Bench for Security** es la herramienta oficial y recomendada que facilita esta tarea.

I. Introducción a Docker Bench for Security

Docker Bench for Security es una herramienta oficial de Docker diseñada para automatizar la auditoría de seguridad de un entorno contenerizado.

Propósito y Mecanismo

- **Definición:** No es más que un *script* que realiza un chequeo exhaustivo de múltiples buenas prácticas de seguridad aplicables a Docker.
- **Alcance de la Auditoría:** La herramienta audita la seguridad en varios niveles del sistema:
 - El **Host** (Sistema Anfitrión).
 - El **Docker Daemon** (Servicio).
 - Los **Ficheros de Configuración** de Docker.
 - Las **Imágenes y Contenedores** en ejecución.
- **Estándar de Referencia:** Las comprobaciones y chequeos se basan en el **CIS Docker Benchmark v1.6.0** (Center for Internet Security). Este *benchmark* es una serie de guías y buenas prácticas de seguridad creadas por la comunidad y es el estándar de la industria para la fortificación de entornos Docker.

Requisitos de Ejecución y Privilegios

La herramienta debe ejecutarse con **privilegios elevados** y un aislamiento reducido para poder inspeccionar a fondo el sistema, el *kernel* y el *daemon* de Docker. Para ello, utiliza:

- **Espacios de Nombre Compartidos:** Debe montar los *namespaces* del *host* (red, procesos, *user namespace*).
- **Capacidades Elevadas:** Se le asignan capacidades adicionales para darle el control necesario para auditar cómo está configurado Docker y el sistema.

II. Procedimiento Práctico: Ejecución de Docker Bench

La forma más conveniente de ejecutar la herramienta es utilizando Docker Compose, lo que encapsula la complejidad de los múltiples parámetros de *runtime* requeridos.

Paso 1: Clonación del Repositorio

Se obtiene el código fuente de la herramienta desde su repositorio oficial.

```
git clone https://github.com/docker/docker-bench-security
```

Propósito: Descargar el *script* de auditoría y los archivos de configuración asociados, como el `docker-compose.yml`.

Paso 2: Ejecución de la Auditoría con Docker Compose

El `docker-compose.yml` preconfigurado en el repositorio maneja todos los complejos parámetros de seguridad necesarios (montaje de *sockets*, uso de *namespaces* del *host*).

Ejecución del Compose: Se utiliza el comando `docker compose up` desde el directorio clonado.

```
docker compose up
```

1. *Propósito*: Docker Compose levanta el servicio, que consiste en ejecutar el *script* de auditoría dentro de un contenedor temporal con los privilegios requeridos.
2. **Análisis de la Salida (Reporte de Auditoría)**: La herramienta muestra en tiempo real una serie de chequeos clasificados en **Warnings** (advertencias), **Pass** (aprobado) y **Fail** (fallido/no conforme), organizados por área:
 - **Configuración del Host**: Chequea la configuración del sistema anfitrión, como la existencia de auditoría en los ficheros y directorios de Docker.
 - **Configuración del Daemon**: Monitoriza y chequea la configuración del servicio de Docker.
 - **Ficheros de Configuración de Docker Daemon**: Revisa la seguridad y los permisos de los archivos de configuración.
 - **Imágenes de Contenedores y Build Files**:
 - **HEALTHCHECK**: Reporta las imágenes que **no tienen definido un chequeo de salud (HEALTHCHECK)**, lo cual es una vulnerabilidad de disponibilidad.
 - **Content Trust**: Chequea si la confianza de contenido está habilitada (un mecanismo para asegurar que solo se descarguen imágenes de sitios o usuarios de confianza).
 - **Uso de COPY en lugar de ADD**: Señala malas prácticas, como el uso de **ADD** cuando **COPY** es suficiente.
 - **Contenedores**: Audita los contenedores en ejecución o detenidos en el sistema.
 - **Security Operations**: Se enfoca en otros aspectos de seguridad del sistema relacionados con Docker.
 - **Docker Swarm**: Revisa las configuraciones de seguridad del orquestador de Docker (aunque su uso es obsoleto, la comprobación se mantiene).

III. Conclusiones y Relevancia en Ciberseguridad

3.1. Importancia de la Auditoría Frecuente

- **Detección de Desviación:** Las buenas prácticas de seguridad son numerosas, y es fácil que se produzcan olvidos o desviaciones de la configuración con el tiempo (creación de nuevas imágenes, *containers* o servicios).
- **Mitigación Continua:** Auditar los sistemas de forma frecuente es esencial. La auditoría no es útil sin la posterior **mitigación** activa de las vulnerabilidades y fallos de configuración reportados.

3.2. Buenas Prácticas Generales (Resumen de Seguridad)

La auditoría refuerza la necesidad de seguir las conclusiones de seguridad vistas previamente:

- **Protección del Daemon:** Proteger el *Docker Daemon* (API REST) es crucial para prevenir el compromiso del *host*.
- **Fortificación del Host:** Mantener el sistema anfitrión actualizado, bien configurado y aplicando técnicas de fortificación.
- **Imágenes Seguras:** Mantener las imágenes **limpias y lo más pequeñas posible** para reducir los vectores de ataque.
- **Mínimos Privilegios:** No ejecutar contenedores como **root** y asegurarse de que los servicios se ejecuten con usuarios dedicados (*non-root*).
- **Disponibilidad:** Definir puntos de salud (**HEALTHCHECK**) para que Docker pueda reportar cuando un contenedor no está funcionando correctamente.

Estrategias de Limpieza y Recuperación de Espacio en Docker

En entornos de ciberseguridad, donde se prueban y construyen numerosas imágenes y contenedores (muchos de ellos temporales), la acumulación de artefactos no utilizados puede consumir rápidamente el espacio en disco del *host*. La **limpieza (*cleanup*)** periódica del sistema Docker es esencial para el mantenimiento, el rendimiento y la disponibilidad del *host*.

I. Impacto de los Artefactos de Docker en el Host

Cada operación de Docker almacena datos en el sistema de ficheros del *host*:

1. **Contenedores Detenidos:** Aunque no ocupan mucho espacio si no generan muchos datos en su capa R/W, se acumulan y ensucian el sistema.
2. **Imágenes y Capas:** Son los artefactos que consumen más espacio. Las imágenes están compuestas de múltiples capas inmutables.
3. **Capas Huérfanas (*Dangling Images*):** Son capas que ya no están referenciadas por ninguna imagen etiquetada, típicamente porque fueron invalidadas por un cambio en un Dockerfile.
4. **Volúmenes y Redes:** Los volúmenes persisten y pueden contener grandes cantidades de datos generados por aplicaciones, mientras que las redes innecesarias se acumulan.
5. **Caché de Construcción:** La caché utilizada para acelerar el *build* también consume espacio considerable en disco.

II. Limpieza Granular por Artefacto (Comando **prune**)

Docker ofrece el comando **prune** para la limpieza automatizada de recursos no utilizados en cada grupo de gestión (**container**, **image**, **network**, **volume**).

1. Contenedores

Los contenedores detenidos (no *running*) pueden ser eliminados para limpiar el sistema.

- **Listar Contenedores (Todos los Estados):**

```
docker container ls -a
```

```
# o su alias
```

```
docker ps -a
```

- **Eliminar Contenedores Detenidos:** El comando `prune` elimina todos los contenedores que han sido parados.

```
docker container prune
```

- *Propósito:* Pregunta confirmación, lista los IDs de los contenedores eliminados y muestra la cantidad de espacio liberado (ej. 181 MB).

- **Eliminar Individualmente:** Para borrar un contenedor específico, debe estar parado (`stopped`).

```
docker container rm [NOMBRE_O_ID]
```

2. Imágenes

La limpieza de imágenes es crucial para liberar grandes cantidades de espacio.

- **Eliminar Capas Huérfanas (*Dangling*):** El comando `prune` sin argumentos borra todas las capas no referenciadas (`<none>`).

```
docker image prune
```

- *Propósito:* Elimina las capas que quedaron sin asignar tras la invalidación de la caché de un *build*.

- **Eliminar Imágenes No Usadas (Incluyendo Capas Huérfanas):** El *flag* **-a** (all) con **prune** borra todas las imágenes que **no están siendo usadas por ningún contenedor existente** en el sistema.

```
docker image prune -a
```

- *Nota de Regla de Negocio:* Una imagen no puede ser eliminada (**rm**) si hay un contenedor asociado a ella, incluso si ese contenedor está parado. Primero se debe eliminar el contenedor.

3. Redes

La limpieza de redes elimina las redes personalizadas que no tienen contenedores conectados.

- **Sintaxis:**

```
docker network prune
```

- *Propósito:* Elimina las redes creadas (ej. las creadas con **docker compose** o **docker network create**) que no estén asociadas a ningún contenedor. Las redes por defecto (**null**, **host**, **bridge**) se mantienen.

4. Volúmenes

Los volúmenes pueden ocupar espacio considerable y deben ser gestionados, ya que **persisten** incluso después de que los contenedores asociados son eliminados.

- **Eliminar Volúmenes Anónimos No Usados:** El comando **prune** borra los volúmenes sin nombre (anónimos) que no tienen contenedores asociados.

```
docker volume prune
```


- **Eliminar Volúmenes Nombrados Específicos:** Los volúmenes nombrados deben borrarse explícitamente.

```
docker volume rm vol1
```

III. Limpieza Agresiva y Resumen del Sistema

Para la limpieza masiva y la liberación de la caché de construcción, se utiliza el comando `docker system`.

1. Resumen del Uso de Disco (`docker system df`)

El comando `system df` proporciona una visión general del uso del disco por parte de Docker.

- **Sintaxis:**

```
docker system df
```

- *Propósito:* Muestra información detallada sobre el espacio ocupado por imágenes, contenedores, volúmenes locales y, lo más importante, el tamaño de la **caché de construcción**.

2. Limpieza Total del Sistema (`docker system prune`)

El comando `system prune` ofrece la forma más agresiva y completa de limpiar el sistema, consolidando las operaciones de `prune` vistas anteriormente.

- **Limpieza Agresiva (Incluye Caché):** Utilizando el *flag* `-a` (all) con `system prune`, se borran:
 - Todos los contenedores parados.
 - Todas las redes no usadas.
 - Todas las imágenes **sin contenedores** (`-a`).
 - Y, lo más importante, **todos los artefactos de la caché de construcción** (que pueden ocupar varios gigabytes).

`docker system prune -a`

- *Propósito:* Esto recupera la mayor cantidad de espacio posible en el *host*.

IV. Conclusiones

La limpieza es un aspecto crítico de la administración de Docker. Dado que es muy fácil ejecutar y detener contenedores ([a diestro y siniestro](#)), es inevitable que el disco del *host* se llene rápidamente.

- **Mantenimiento Esencial:** Es crucial añadir la limpieza y algún tipo de monitorización al proceso operativo para mantener el sistema lo más limpio posible.
- **Recuperación de Caché:** El comando `docker system prune -a` es la forma más efectiva de recuperar espacio masivo al eliminar la caché de construcción.